PROJECT UPDATE / STAGE 03

SECURE UART COMMUNICATION

Confidentiality + error correction + serial transport

ChaCha20 • Extended Hamming (13,8) • UART

256-bit	secret key
96-bit	nonce
13,8	SECDED code
8N1	UART framing

Stage 3: system integration and end-to-end verification

2026

Stage 3 brings the protection layers together

Move from individually designed modules to a framed, end-to-end secure serial message path.

01

Confidentiality

Encrypt each message byte with a ChaCha20 keystream generated from a shared key, nonce and block counter.

02

Error correction

Apply extended Hamming (13,8) to each packet byte so a single-bit error can be corrected and a double-bit error detected.

03

Serial transport

Frame the packet, buffer outgoing bytes with a FIFO, and send/receive through UART TX and RX.

Design goal **Person B should recover the original variable-length message after UART reception and Hamming decoding.**

Scope note: current RTL uses raw ChaCha20; it does not yet include a Poly1305 authentication tag.

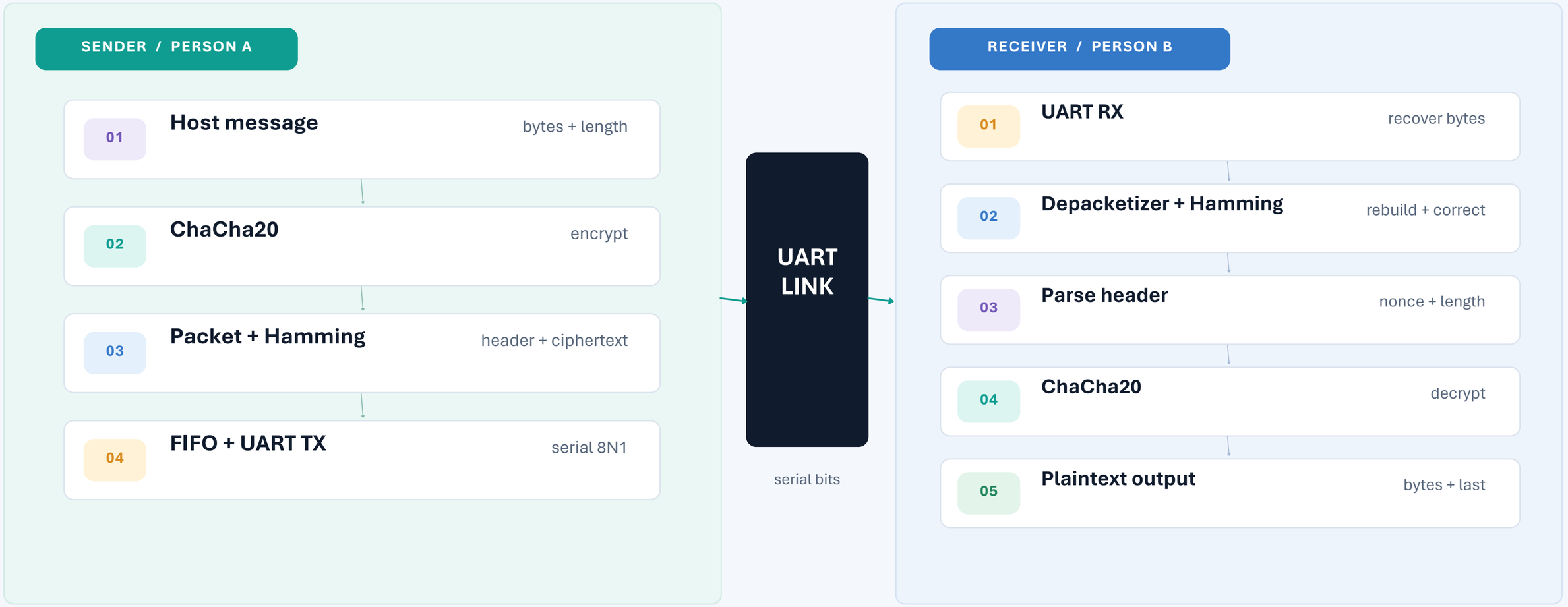
Different layers solve different problems

Keep cryptography, channel-error handling and transport separate so each module has a clear responsibility.

APPLICATION **Message + length**	Defines what is sent and where the message ends.
CRYPTOGRAPHY **ChaCha20**	Protects confidentiality; keystream XOR makes encryption and decryption symmetric.
ERROR CONTROL **Hamming (13,8)**	Adds redundancy for single-bit correction and double-bit detection.
TRANSPORT **UART 8N1**	Serializes bytes with start/data/stop framing; baud generator sets bit timing.

End-to-end data path

The transmitter adds protection before serialization; the receiver reverses the same layers in the opposite order.

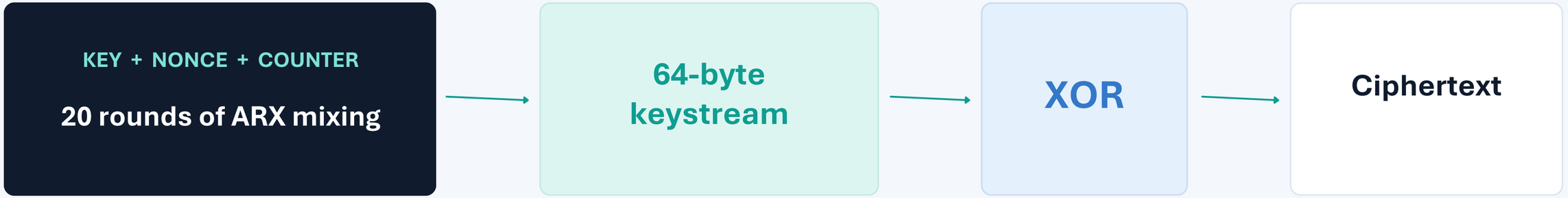


Packet format is defined on slide 8. The nonce is sent in clear form but is Hamming-protected.

ChaCha20: generate a keystream, then XOR

ChaCha20 is a stream cipher. It supports arbitrary message lengths by generating successive 64-byte keystream blocks.

256 bits	96 bits	32 bits	64 bytes
KEY	NONCE	BLOCK COUNTER	PER KEYSTREAM BLOCK



Encryption: $C = P \oplus S$

Decryption: $P = C \oplus S$

The same key + nonce + counter sequence regenerates the same keystream. Never reuse a nonce with the same key for a different message.

ChaCha20 alone provides confidentiality, not message authentication. Current RTL does not append a Poly1305 tag.

Extended Hamming (13,8): one codeword per byte

Eight data bits are combined with four Hamming parity bits and one overall parity bit.

1	2	3	4	5	6	7	8	9	10	11	12	13
PARITY	PARITY	DATA	PARITY	DATA	DATA	DATA	PARITY	DATA	DATA	DATA	DATA	OVERALL

Parity positions

1, 2, 4, 8

Data positions

3, 5, 6, 7, 9, 10, 11, 12

Overall parity

Position 13

SECDED — Corrects any single-bit error • Detects any double-bit error

Codeword positions are numbered 1–13; position p maps to codeword[p−1]. Based on information encoding and Hamming-distance principles [1].

A 13-bit codeword becomes two UART payload bytes

The packetizer preserves all 13 codeword bits and pads the unused three bit positions with zeros.

HAMMING CODEWORD



UART BYTE 1

codeword[7:0]

UART BYTE 2

000 | codeword[12:8]

13 meaningful code bits

3 padding bits must remain zero

Each UART byte is then serialized as an 8N1 frame: start bit + 8 data bits + stop bit.

Packet format makes variable-length messages unambiguous

The receiver first recovers the header, then uses the nonce and length to process exactly the ciphertext bytes.

SYNC	NONCE	LENGTH	CIPHERTEXT
C3 5A	96-bit value	N (big-endian)	N bytes
2 bytes	12 bytes	2 bytes	variable

Nonce construction in current top-level design

nonce = { 64-bit prefix , 32-bit message counter }

A new nonce is used for each encryption under the same key.

Nonce safety rule

Never repeat a key + nonce pair for two different messages.

The nonce is not secret. Preserve counter uniqueness across resets (or provision a fresh prefix/key); raw ChaCha20 does not include an authentication tag.

What happens to “Hello, I am Rishi!”?

18 ASCII bytes pass through encryption, framing, Hamming coding and UART serialization.

18	18	34	68
PLAINTEXT BYTES	CIPHERTEXT BYTES	PACKET BYTES	UART PAYLOAD BYTES

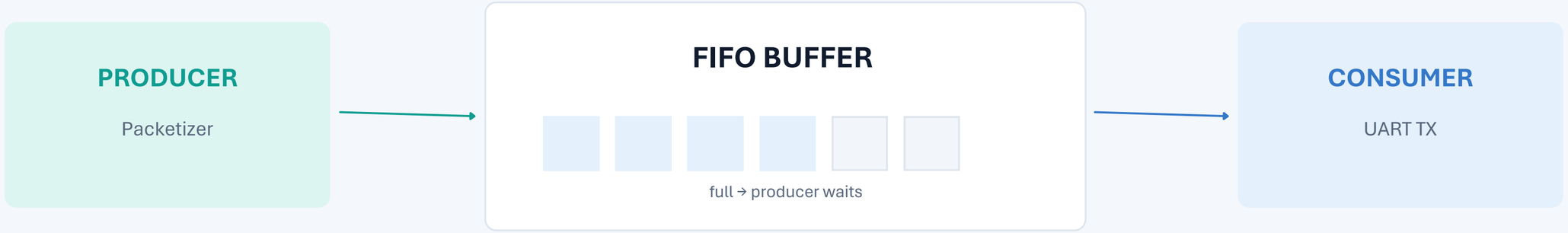
Packet composition	16-byte header + 18-byte ciphertext = 34 bytes
Hamming encoding	34 codewords × 13 bits = 442 protected bits
Two-byte packetization	34 codewords × 2 UART bytes = 68 UART bytes
UART 8N1 line cost	68 frames × 10 serial bits = 680 bit periods



The 3 padding bits in every second UART payload byte are not part of the 13-bit Hamming codeword.

FIFO + ready/valid handshakes keep the byte stream ordered

The producer may run ahead of the UART consumer by the FIFO depth, but it must stop when the buffer is full.



TRANSFER RULE

valid AND ready = byte accepted on the clock edge

If valid = 1 and ready = 0, hold data stable. Advance only after the transfer.

Producer–consumer coordination and FIFO buffering are discussed in MIT 6.004, Lecture 19.1 [2].

RTL modules and their responsibilities

The top-level controller coordinates the existing modules; each module retains a focused function.

TRANSMIT PATH

chacha20_encrypt.v	XOR plaintext with generated keystream
hamming_encoder.v	Map 8 data bits to a 13-bit SECDED codeword
hamming_packetizer.v	Split each codeword into two UART bytes
fifo.v + uart_tx.v	Buffer bytes and serialize 8N1 frames
baud_gen.v	Generate baud timing ticks

RECEIVE PATH

uart_rx.v	Recover each serial UART byte
hamming_depacketizer.v	Rebuild 13-bit codewords from byte pairs
hamming_decoder.v	Correct / detect supported bit errors
packet parser	Read sync, nonce and payload length
chacha20_decrypt.v	XOR ciphertext with the matching keystream

Shared keystream engine: chacha20_keystream.v is instantiated by both encryption and decryption wrappers.

Verification moves from module tests to loopback integration

Use deterministic messages and check the plaintext bytes returned by the receiver, not only internal ciphertext signals.

5 bytes — “Hello”

Short-message framing

18 bytes — “Hello, I am Rishi!”

Representative text payload

70 bytes — Pattern payload

Crosses the 64-byte ChaCha20 block boundary

Component-level checks

- UART / FIFO modules have dedicated tests.
- Hamming encoder and decoder testbenches cover parity and error cases.
- ChaCha20 has vector and multi-block round-trip tests.

Top-level integration

- tb_secure_uart_top.v is prepared for UART loopback.
- Checks 5-, 18- and 70-byte recovery and message boundaries.
- Run compile + simulation locally before claiming end-to-end pass.

Status shown here is based on the project files and checks discussed so far; end-to-end simulation remains a required final step.

What is implemented — and what remains

Stage 3 establishes the data path; security hardening and fault-injection tests are the next engineering steps.

Current design

- ChaCha20 confidentiality layer
- Hamming (13,8) SECDED layer
- Length-prefixed UART packet format
- Counter-based nonce input to decryption

Known limitations

- Raw ChaCha20 has no authentication tag
- Nonce counter must not repeat after reset
- Sync/length parser is not a full recovery protocol
- Combinational 20-round block path may limit timing

Next steps

- Run the full top-level testbench
- Inject serial bit errors and verify correction
- Add ChaCha20-Poly1305 authentication
- Synthesize and assess timing / area

Hamming protects against accidental bit errors; authentication is needed to detect deliberate tampering.

Suggested five-person work split

Keep module ownership clear, but make integration and verification a shared responsibility.

01

ChaCha20 encryption

Encrypt wrapper, keystream interface and known-vector tests

02

ChaCha20 decryption

Decrypt wrapper, nonce/counter consistency and round-trip tests

03

Hamming encoder + packetizer

13-bit codeword generation and two-byte packet mapping

04

Hamming decoder + depacketizer

Codeword reconstruction, syndrome checks and error tests

05

UART integration + verification

Top-level controller, FIFO flow, packet framing and end-to-end TB

The role split is a proposal; final ownership can be adjusted to the team’s workload.

References and further reading

The slide-level reference numbers point to the learning material used for encoding, buffering and UART design.

[1] **MIT OpenCourseWare — 6.004, 1.1 Annotated Slides: Basics of Information**

Covers information encoding, Hamming distance, parity, error detection and error correction.

[2] **MIT OpenCourseWare — 6.004, 19.1 Annotated Slides: Concurrency and Synchronization**

Includes synchronous communication, FIFO buffering, producer–consumer coordination and flow control.

[3] **B. Roy Dakua, M. I. Hossain and F. Ahmed — Design and Implementation of UART Serial Communication Module Based on FPGA (ICMEIE 2015)**

FPGA-oriented UART implementation using Verilog HDL; deposited on Zenodo.

[4] **IEEE Xplore PDF — link supplied for Stage 3 reference**

The supplied IEEE link is included as provided; publication title/metadata could not be reliably resolved during preparation.

URLs are clickable in the PowerPoint. The IEEE reference is preserved exactly as provided.

STAGE 3 SUMMARY

A layered approach to reliable serial messaging.

ChaCha20 protects confidentiality. Hamming adds error correction. UART moves the framed bytes between endpoints.

Thank you